

基于一致性哈希和随机选取的PBFT算法改进

翟社平^{1,2}, 霍媛媛¹, 杨锐¹, 聂浩楠¹

1. 西安邮电大学 计算机学院, 西安 710121

2. 西安邮电大学 陕西省网络数据分析与智能处理重点实验室, 西安 710121

摘要:针对实用拜占庭容错算法PBFT存在的系统动态性不足以及主节点选取随意导致的共识效率较低、系统稳健性较差等问题,提出一种基于一致性哈希和随机选取的CRPBFT共识算法。采用一致性哈希对节点进行分组,在分组的基础上增加节点动态变化机制,为系统提供动态的网络结构。根据节点在共识中的表现动态计算各节点的信誉值,同时定义主节点候选列表、普通节点和恶意节点这三种节点信誉层次,从高信誉值的主节点候选列表中使用可验证随机函数选取可靠且身份难以被恶意预测的主节点,并将符合信誉值要求的节点组成较稳定的共识集群。实验结果表明CRPBFT算法较PBFT算法中共识节点集群的可靠程度更高,在共识时延、吞吐量以及系统稳健性方面的性能优于PBFT算法。

关键词:区块链;信誉机制;可验证随机函数;实用拜占庭容错算法

文献标志码:A **中图分类号:**TP311.5 **doi:**10.3778/j.issn.1002-8331.2302-0225

Improvement of PBFT Algorithm Based on Consistent Hash and Random Selection

ZHAI Sheping^{1,2}, HUO Yuanyuan¹, YANG Rui¹, NIE Haonan¹

1. School of Computer Science, Xi'an University of Posts and Telecommunications, Xi'an 710121, China

2. Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Xi'an University of Posts and Telecommunications, Xi'an 710121, China

Abstract: Aiming at the problems of the practical Byzantine fault-tolerant algorithm (PBFT), such as insufficient system dynamics, low consensus efficiency and poor system robustness caused by the random selection of master nodes, a consensus algorithm of CRPBFT based on consistent hash and random selection is proposed. Firstly, the nodes are grouped by consistent hash, and the dynamic change mechanism of nodes is added to provide a dynamic network structure for the system. Secondly, the reputation value of the node is dynamically calculated according to the performance of the node in the consensus. At the same time, this paper defines three node reputation levels, namely, the candidate list of primary nodes, common nodes and malicious nodes. The primary node that is reliable and whose identity is difficult to be maliciously predicted, is selected through the verifiable random function, and the nodes that satisfied the reputation value requirements are selected to form a relatively stable consensus cluster. Experimental results show that CRPBFT algorithm is more reliable than consensus node cluster in PBFT algorithm, and its performance in consensus delay, throughput and system robustness is better than PBFT algorithm.

Key words: blockchain; reputation mechanism; verifiable random function; practical Byzantine fault-tolerant algorithm

自2009年比特币系统^[1]正式发布以来,作为其底层技术的区块链也逐渐得到研究者的广泛关注。区块链技术是在经济学、密码学及计算机科学等技术上发展而

来的^[2],能够在没有可信中心的系统环境下实现多方对等可信的信息传输^[3],在数字金融、物联网、供应链管理等领域的应用得到了广泛的研究^[4]。

基金项目:国家自然科学基金(61373116);工业和信息化部通信软科学项目(2018-R-26);陕西省教育厅科学研究计划项目(18JK0697);陕西省社会科学基金(2016N008);陕西省重点研发计划项目(2022GY-038);西安市社会科学规划基金(17X63);陕西省大学生创新创业训练计划项目(202211664016)。

作者简介:翟社平(1971—),男,博士,教授,CCF高级会员,研究方向为区块链、语义计算;霍媛媛(1998—),通信作者,女,硕士研究生,研究方向为区块链技术,E-mail:1341544022@qq.com;杨锐(1976—),女,硕士,讲师,研究方向为区块链;聂浩楠(1998—),男,硕士研究生,研究方向为区块链技术。

收稿日期:2023-02-17 **修回日期:**2023-06-29 **文章编号:**1002-8331(2024)12-0294-09

共识算法是区块链技术的核心要素,其性能直接影响区块链系统中的交易时延、吞吐量、可扩展性及系统稳定性等性能^[5]。区块链根据对系统接入节点身份的审查机制不同可分为非许可链和许可链。非许可链即为公有链,典型共识算法有工作量证明(proof of work, PoW)、权益证明(proof of stake, PoS)和授权股份证明(delegated proof of stake, Dpos)等^[6-8]。许可链包括联盟链及私有链,典型共识算法为实用拜占庭容错算法(practical Byzantine fault tolerance, PBFT), PBFT算法通过三阶段的共识过程保证节点间数据的一致性,为系统提供了 $(n-1)/3$ 的容错性^[9]。

但是PBFT算法无法支持节点的动态变化,其区块链网络结构静态,仅支持静态网络下一致性协议的执行,节点的动态变化将导致系统中数据的映射地址失效,因此需要重启系统来恢复算法的正常运行。当系统节点数量较多且存在拜占庭节点的情况下,节点的动态变化将会消耗大量网络资源,系统难以高效运行,严重影响了系统的可用性。另外PBFT算法依赖于弱同步网络环境的假设,系统需要通过定时器来规定网络传输延迟的范围。PBFT中主节点负责区块的生成,当主节点在规定时间内没有对消息进行有效处理导致超时或者存在恶意行为时,将会触发视图切换协议,造成网络资源的浪费。然而PBFT算法中主节点的选取策略随意,通过对视图编号进行简单模运算依次选取主节点,导致恶意节点当选主节点的可能性较大,且攻击者易根据视图编号预测选主结果从而对主节点进行攻击^[10],频繁的视图切换会浪费系统资源,影响共识的效率及系统稳健性。

针对PBFT网络结构静态的问题,黄保华等^[11]为参与共识的节点集合增设候补节点集,设置节点的替换规则实现共识节点的动态替换。徐治理等^[12]对PBFT共识的检查点协议进行了扩展,增加了对节点动态增删的功能。黄冬艳等^[13]将系统节点进行分片处理,提出一种分组共识机制RBFT,组间通过PBFT算法共识,组内采用Raft共识,利用Raft算法特性提供系统节点的动态变化能力。为解决PBFT主节点选取随意的问题,卢丽等^[14]通过一致性空间来优化主节点的选取方式,将副本节点映射至一致性哈希空间上,通过定期旋转虚拟节点(桶)来为系统分配正确的主节点。Yang等^[15]提出了节点间决策广播模型和阈值投票计数模型,通过对节点进行联合故障分析以确保共识的安全可靠与容错性。Lao等^[16]面向物联网应用环境提出了G-PBFT算法,利用物联网设备较为固定的属性,选取更忠诚强健的物联网设备作为主节点,提高主节点的可靠性。陈宇等^[17]基于无人机的应用场景,将共识节点分组,选取各组内部分高信誉值节点组成本组的委员会,剩余节点作为共用节点,通

过分层的结构,将主节点隐匿于委员会中,降低了主节点被敌手攻击的风险。Tong等^[18]提出的Trust-PBFT共识算法通过设计点对点网络中节点的信任计算模型来评估节点的可信度,对符合PBFT共识参与节点的资格进行判定,确保参与共识节点的可靠性。

上述研究内容都从不同角度解决PBFT算法存在的相关问题,然而改进方面都较为单调没有综合考虑选取结果的集中化趋势与主节点身份的安全性。本文在PBFT算法的基础上,提出一种基于一致性哈希和随机选取的共识算法(consistent hash and random selection practical Byzantine fault tolerance, CRPBFT)。引入一致性哈希环对节点进行分组,解决系统动态性不足的问题。设计信誉模型对分组内节点进行区分,并结合可验证随机函数改进主节点的选取方式,避免主节点被提前预测攻击,降低主节点为恶意节点的概率,缓解主节点选取的集中化趋势,提高共识效率及系统稳健性。

1 相关知识

1.1 PBFT 算法

PBFT算法将拜占庭容错算法指数级的高通信复杂度降低到多项式级,为拜占庭容错类算法真正应用于工程领域提供了可能^[19]。PBFT算法由一致性协议、视图转换协议以及检查点协议组成,一致性协议用来完成系统节点间的全网数据共识;当主节点作恶时,启动视图转换协议重新选取主节点;检查点协议定期清理日志中的消息以节约系统存储资源。

PBFT共识过程主要包括预准备(pre-prepare)、准备(prepare)和确认(commit)等阶段,共识流程如图1所示,参与共识的节点可以分为客户端Client、主节点Primary以及从节点Replica三种类型,客户端会将消息请求发送给主节点,主节点对消息进行验证,验证通过后向所有从节点广播该请求,从节点对消息进行验证广播,最后将共识结果发送给客户端。

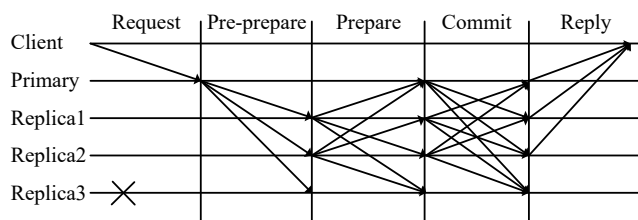


图1 PBFT 共识过程

Fig.1 PBFT consensus process

PBFT中主节点的编号 P_i 通过式(1)计算得出,其中 V 是视图编号, n 为系统的节点数量。当需要更换主节点时,将视图编号 V 加1并代入计算,节点按选取规则轮流当选主节点。

$$P_i = V \bmod n \quad (1)$$

PBFT算法的主节点选取规则随意,恶意节点被选为主节点的可能性较高,且攻击者可能会根据视图编号等相关信息对主节点身份进行预测攻击,影响系统的稳健性与共识效率。

1.2 一致性哈希算法

一致性哈希算法是1997年由Karger等人提出的,用来解决分布式系统中数据存储存在的问题^[20]。一致性哈希算法较为高效地解决了分布式系统中节点动态增删时带来的存储负载不均的问题,使分布式系统对于部分数据的热点访问具有良好的动态适应性和效率。一致性哈希算法将整个哈希空间映射为一个虚拟的哈希环,哈希空间范围从 $0 \sim 2^m$ 按顺时针方向进行组织。一致性哈希将存储节点通过哈希运算映射到哈希环上,再将数据资源以同样的方式进行映射,并按照顺时针方向为数据资源定位其对应的存储节点,完成存储节点与数据资源的关系映射。为更好地解决负载不均衡的问题,引入节点的虚拟节点,虚拟节点关联的数据资源将由其对应的节点进行处理。相比于传统固定哈希的映射方式,当哈希环上节点的映射关系发生变化时,一致性哈希并不需要对所有节点及其对应数据的映射关系进行重新定位,只需对发生映射关系变化节点的相邻节点上的数据进行局部的迁移变化,极大地降低了对现有节点及数据间映射关系的变动,节省了系统的资源。本文通过一致性哈希算法对系统节点进行分组,为PBFT提供了节点动态变化的能力。

1.3 可验证随机函数

可验证随机函数(verifiable random function, VRF)是1999年由Silvio等提出的一种具有非交互零知识证明特点的伪随机数生成函数^[21]。将使用者的私钥SK和随机种子Rand_seed作为函数的输入,即可得到一个伪随机数Rand_num和证明Proof,Proof的作用就是证明Rand_num是由某个使用者产生的,所有人都可以通过使用者的公钥PK以及Proof在不泄露使用者私钥的条件下完成对伪随机数生成者的身份验证。VRF函数具有可验证性、唯一性及随机性的特点,其中可验证性指通过PK和Proof可以验证Rand_num生成者的身份,唯一性指对于相同的输入输出也是相同的,随机性指对于攻击者,VRF生成的伪随机数和随机数不可被区分。可验证随机函数VRF的流程如下:

(1)节点使用加密算法生成一对密钥,包括私钥SK和公钥PK。

(2)输入私钥SK和随机种子Rand_seed,通过VRF()函数得到输出结果,伪随机数Rand_num和用于身份证明的Proof。

$$VRF(Rand_seed, SK) = (Rand_num, Proof) \quad (2)$$

(3)VRF_Ver()函数用于其他节点对产生伪随机数的节点进行验证,输入产生伪随机数节点的公钥PK、随机种子Rand_seed、伪随机数Rand_num,以及证明Proof,输出为True或者False。

$$VRF_Ver(PK, Rand_seed, Rand_num, Proof) \quad (3)$$

2 CRPBFT 共识算法

2.1 一致性哈希分组策略

通过一致性哈希算法对系统节点进行分组,当节点动态变化时,能够尽可能小地改变节点与分组的映射关系,不需要重启区块链系统。节省了网络开销,提升了系统的动态性及可用性。

2.1.1 初始化分组

分组过程中为了减少分组结果倾斜,引入虚拟节点机制使得分组结果更加均衡。同时设置各分组内节点数量的下限阈值,以保证组内节点分类的合理性。系统初始化分组过程如下:

(1)首先确定分组数量N,通过哈希算法求得每个分组编号对应的哈希值,然后根据哈希值将其映射到哈希环上,确定各分组所在位置。同时定义分组的虚拟分组,通过多次哈希计算,确定各分组在哈希环上对应的虚拟分组位置,最终每个分组在哈希环上均有多个对应的虚拟分组存在。一致性哈希分组策略如图2所示,以分组1及其对应的虚拟分组举例,分组1(1)、分组1(2)均为分组1在哈希环上的虚拟分组,用以在分组时均衡分组结果。

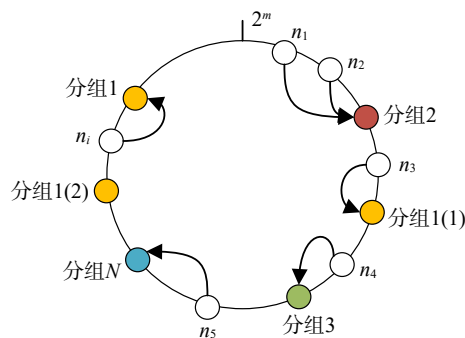


图2 一致性哈希分组

Fig.2 Consistent hash grouping

(2)根据节点进入系统的先后顺序对节点进行编号,将各节点编号i及节点经过哈希运算的次数count作为哈希算法的输入参数,通过 $hash(i + count)$ 将所有系统节点映射到哈希环上。然后在哈希环上按照顺时针方向为节点分配分组,各节点位置后遇到的第一个分组位置即为每个节点所在分组。图2中节点n3的分组过程如下:沿顺时针方向确定节点n3对应的分组位置为虚拟分组1(1),再由虚拟分组1(1)确定对应的实际分组为分组1。

(3)对每组节点数量进行验证,若不满足分组节点数量的下限阈值,则按照分组规则重新进行分组。分组结束后,系统节点被划分至不同的分组,每个分组中都包含系统中的部分节点集合,各节点在本地维护全网节点列表及所在分组的节点列表。

2.1.2 节点动态变化机制

节点动态变化机制包括节点的动态加入以及动态退出区块链网络。当节点动态变化时,由于采用了一致性哈希对节点进行分组,只需在一致性哈希环上改变某一分组内的节点映射关系,不需要重启区块链网络,为系统提供了动态的网络结构。

(1)节点动态加入机制

图3为节点动态加入流程,其中 Replica-new 为新加入节点,Supervision-node₀为新增节点对应拟分配分组中的监督节点,Replica₁~Replica₃为该分组的组内节点,Supervision node₁~Supervision-node₃为其余分组的监督节点。

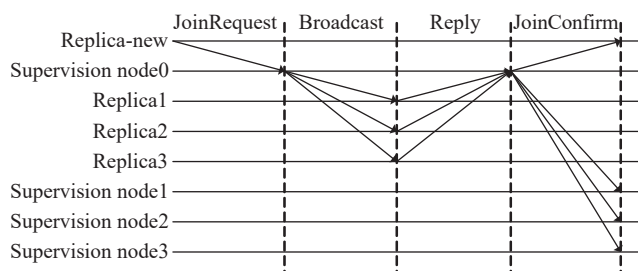


图3 节点动态加入过程

Fig.3 Dynamic joining process of nodes

节点动态加入的具体步骤如下:

映射阶段:经过授权的节点 Replica-new 先通过哈希运算映射出其在一致性哈希环上的位置,确定拟对应的分组编号。

申请阶段:节点 Replica-new 向该分组的监督节点(监督节点在2.2节详细介绍)发送加入请求 JoinRequest 消息,包括节点的签名信息及时间戳内容。

验证阶段:当组内监督节点收到节点 Replica-new 发送的 JoinRequest 请求时,先对其身份进行验证,验证通过后将 JoinRequest 消息广播至其所在分组。组内成员对该消息进行投票,并将签名后的投票结果发送至组内的监督节点,当组内监督节点收到超过组内成员半数的同意加入分组投票后,组内对该节点的加入请求达成一致,同意节点 Replica-new 加入分组。

确认阶段:节点 Replica-new 所在分组的监督节点向节点 Replica-new 以及其余分组的监督节点发送节点加入 JoinConfirm 消息。其余分组的监督节点收到加入确认消息后将其广播至所在分组,此时全网节点对节点 Replica-new 的加入达成共识,各节点更新系统节点总数 M 和全网节点列表。

(2)节点动态退出机制

图4为节点动态退出流程,其中 Replica-exit 为申请退出节点。

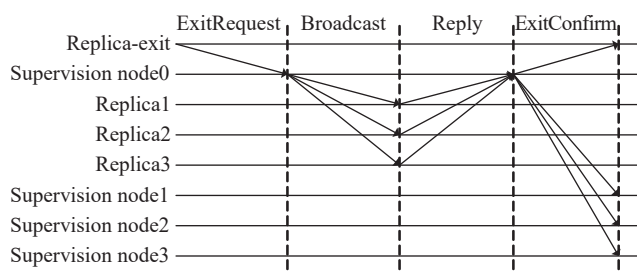


图4 节点动态退出过程

Fig.4 Dynamic exit process of nodes

其具体步骤如下:

申请阶段:当节点 Replica-exit 需要退出网络时,向该分组的监督节点提交退出请求 ExitRequest。若退出节点为监督节点,则重新选取该分组监督节点后,才可申请退出网络。

验证阶段:当组内监督节点收到节点 Replica-exit 发送的 ExitRequest 请求时,对消息进行验证,验证通过后将 ExitRequest 消息广播至其所在分组。组内成员对该消息进行投票,并将签名后的投票结果发送至组内的监督节点,当组内监督节点收到超过半数的同意退出分组投票后,组内对该节点的退出请求达成一致,同意其退出分组。

退出阶段:节点 Replica-exit 所在分组的监督节点向节点 Replica-exit 以及其余分组的监督节点发送节点退出 ExitConfirm 消息。其余分组的监督节点将退出确认消息广播至所在分组,此时全网节点对节点 Replica-exit 的退出达成共识,完成该节点的删除,各节点更新系统节点总数 M 和全网节点名单。若节点的退出导致组内节点数量小于下限阈值,则需要按照分组规则重新分组,更新各节点所在分组的节点列表信息。

2.2 动态信誉模型

通过构建动态信誉模型区分诚实节点及恶意节点,根据各节点在共识过程中的表现计算节点信誉值,并从高信誉值节点中选取主节点,降低主节点作恶概率,提高共识效率。

2.2.1 节点角色划分

各分组节点按照信誉值可被分为三个信誉层次,如图5所示,第一层为主节点的候选列表,由分组内前 H_1 个高信誉值节点组成,第二层由普通节点组成,第三层为恶意节点,系统的共识节点集群由各分组内第一层及第二层中的节点构成,共同完成区块的共识过程。

系统内包括主节点、监督节点、普通节点、恶意节点这几种角色。

主节点:主节点从各组高信誉值的主节点候选列表中选出,负责接收验证客户端的请求,并将数据打包生

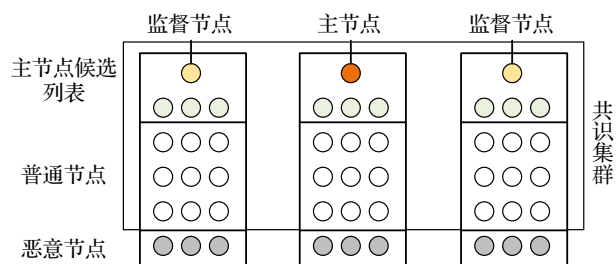


图5 CRPBFT算法节点分类

Fig.5 Node classification of CRPBFT algorithm

成新区块,每轮共识中仅有一个主节点。若主节点在共识过程中出现宕机或作恶行为,则对其进行信誉惩罚,并触发视图切换协议选取新的主节点,失效主节点信誉值更新后重新确定其所属的信誉层次。

监督节点:每个分组均设置了监督节点,主要负责处理对应分组内节点的动态加入或退出,节点信誉值的动态更新以及向恶意节点同步共识结果。另外主节点所在分组,其组内的监督节点由主节点担任。若某分组监督节点宕机或存在恶意行为,则选取该组主节点候选列表中信誉值最高的节点为新的监督节点。

普通节点:普通节点和主节点候选列表内的节点作为共识节点集群,参与共识过程以确保系统数据的一致性。主节点候选列表中的节点可能会由于恶意行为导致信誉值降低,若降低后的信誉值不在分组前 H_1 个高信誉值排名内,但却高于恶意节点层次的信誉阈值时,将被划分到普通节点层次中。

恶意节点:节点信誉值过低时,将被认为是恶意节点,不允许参与共识过程,只能在本地同步最后的共识结果。当共识集群中的节点受到信誉惩罚导致其信誉值小于恶意节点层次的信誉阈值时,将被划分为恶意节点,失去参与共识的资格。

2.2.2 信誉值计算模型

节点的信誉值有利于对节点能否成功完成共识的能力进行衡量,激励节点诚实参与共识,区分诚实与恶意节点,提高共识效率。系统中较高信誉值的节点作恶可能性更小,主节点及监督节点将从高信誉值节点中选取,当节点信誉值过低时,将被认为是恶意节点,无法参加共识过程。

CRPBFT算法将节点在共识过程中成功完成共识的次数 Con_suc_i 与节点参与的总共识次数 Con_all_i 的比值 C_i 以及节点完成区块记账的时间 T_i 作为影响节点信誉值的两项指标,节点在共识过程中的行为将对其信誉值产生影响。节点的 C_i 越接近于1,则节点共识成功率越高,具有较好的稳定性和信任度。定义式(4)来计算 C_i 指标对节点信誉值的贡献,其中 α 影响 λ_1 函数图像的递增速度, λ_1 随着 C_i 的增大而递增,符合 C_i 对信誉值影响的正相关特性。 T_i 指标小的节点具有更好

的性能,节点所获得的信誉值越高。定义式(5)来计算 T_i 指标对节点信誉值的影响,其中 β 影响 λ_2 函数图像的递减速度, λ_2 随着 T_i 的增大而递减,呈负相关特性。

$$\lambda_1 = 1 - e^{-\alpha C_i} \quad (4)$$

$$\lambda_2 = e^{-\beta T_i} \quad (5)$$

融合上述影响节点信誉值的两项指标,节点信誉值的计算公式如式(6)所示,式中 a 代表对节点共识表现的激励程度。

$$a(\lambda_1 + \lambda_2) \quad (6)$$

各节点的信誉值初始化为0,定义 R_i 为节点 i 在第 t 轮共识获得的信誉值, R_{t-1} 为上一轮共识后获得的信誉值,则节点信誉值的更新规则如式(7)所示。当节点出现拜占庭行为时,将其信誉值减少50%。若节点拜占庭行为过多导致信誉值小于设定阈值时,将节点记录为恶意节点,取消其参加共识的资格。

$$R_t = R_{t-1} + a(\lambda_1 + \lambda_2) \quad (7)$$

每轮共识结束后由各分组的监督节点负责统计相关信息并计算节点的信誉值更新结果,并将更新后的信誉值发送给组内节点,组内节点验证无误后,完成信誉值的更新。

2.3 可验证随机函数选主策略

通过在高信誉值的主节点候选列表中采取可验证随机函数VRF进行选主,有效地解决了PBFT主节点选取随意带来的问题。主节点选取时高信誉值条件的设定有效地降低了主节点为恶意节点的概率。同时随机选取的方式能够避免恶意节点提前预测主节点身份对其进行攻击,导致共识失败。结合节点动态信誉的更新机制,随机选取主节点的方式也能够使主节点候选列表中的节点均有当选主节点的可能,缓解了选主过程的集中化趋势,避免出现马太效应的同时激发节点参与共识的积极性,提高系统的共识效率。

可验证随机函数VRF选主流程如图6所示,由客户端负责完成共识主节点及各组监督节点的身份选取。具体步骤如下:

(1)客户端向各分组的主节点候选列表发送用于主节点选取的随机种子 $Rand_seed$,发送后启动定时器,若在规定时间内仍未收到某分组的身份选取结果,则重发随机种子,避免接收不到结果时陷入无意义的等待。

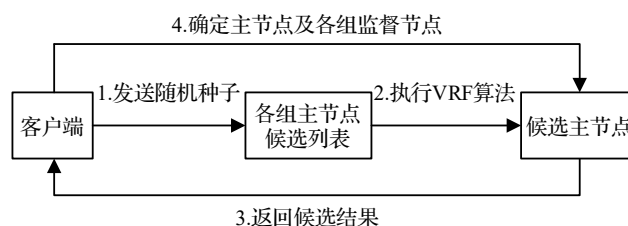


图6 CRPBFT算法选主流程

Fig.6 Primary selection process of CRPBFT algorithm

(2)各分组主节点候选列表内的节点将客户端发送的随机种子 $Rand_seed$ 及节点的私钥 SK 作为可验证随机函数 VRF 身份选取算法的输入,执行算法 $VRF(Rand_seed,SK)=(Rand_num,Proof)$,根据产生的选取结果来判断自己是否为本分组的候选主节点。

(3)若节点为对应分组的候选主节点,则向客户端发送 VRF 的执行结果 $(Rand_num,Proof)$,客户端在定时器规定范围内接收不同分组候选主节点的选取结果,并通过节点身份验证算法 $VRF_Ver(PK,Rand_seed,Rand_num,Proof)$ 对节点的身份选取结果进行检验。

(4)对各分组的候选主节点身份验证通过后,客户端选取候选主节点中信誉值最大的作为主节点,未当选主节点的其余分组候选主节点则被选为对应分组的监督节点。由于在第一轮共识主节点的选取中,节点信誉值均为0,因此客户端将最先收到符合结果的候选主节点选为主节点。

2.4 共识阶段

CRPBFT 算法保持 PBFT 算法的三阶段共识过程,共识流程如下:

(1)Request阶段:客户端向主节点发送请求 $\langle request, o, t, c \rangle$,其中 o 为请求执行的操作, t 为时间戳, c 为客户端身份标识。

(2)Pre-prepare阶段:主节点验证 $request$ 消息的合法性,若验证通过则为该请求分配一个序列号 n ,向所有从节点发送预准备消息 $\langle \langle pre-prepare, V, n, d(m) \rangle, m \rangle$,其中 V 为当前视图编号, $d(m)$ 为消息 m 的摘要。若从节点对预准备消息内容验证通过,则进入准备阶段。

(3)Prepare阶段:从节点互相发送准备消息 $\langle prepare, V, n, d(m), i \rangle$,其中 i 为节点编号。当节点收到超过 $2f+1$ 个其他从节点发送的与预准备消息内容一致的准备消息时,进入确认阶段。

(4)Commit阶段:共识节点间广播确认消息 $\langle commit, V, n, d(m), i \rangle$,当收到超过 $2f+1$ 个验证过的确认消息后,消息 m 被执行且变为 committed 状态。

(5)Reply阶段:节点将请求消息的执行结果 $\langle reply, V, t, c, i, r \rangle$ 返回给客户端,其中 r 为请求消息 m 的执行结果,若客户端收到至少 $f+1$ 个回复消息时即达成共识。

3 实验分析

本文从共识时延、吞吐量和系统稳健性这几个方面对比 PBFT 算法、CRPBFT 算法、CBFT 算法^[11]以及 CPBFT 算法^[12]的性能,以验证 CRPBFT 算法的有效性。设置不同数量节点下四种算法在共识时延和吞吐量上的性能测试对比,在共识时延中的测试中对可验证随机

函数算法运行的时间进行实验测试,分析其对整体共识时延的影响。从视图转换和节点对交易请求的处理情况这两个方面对比分析两者的系统稳健性。

本文实验环境为 Windows 10 操作系统,AMD Ryzen 7 4700U 的处理器,内存为 16 GB,实验基于 Java 语言在超级账本开源项目的基础上搭建区块链系统,并使用 OriginPro 软件绘制实验结果。

3.1 共识时延

共识时延是从客户端发起交易请求到区块完成上链所需的时间间隔。在 CRPBFT 算法中使用可验证随机函数来选取主节点,共识过程中若主节点需重新选取,则可能会影响共识时延。

首先测试运行可验证随机函数算法所需的时间,可验证随机函数主要有生成伪随机数与证明以及验证节点身份两个部分。设置 1 000 次实验对其运行时间进行测试,实验结果如表 1 所示。

表 1 VRF 算法运行时间测试

Table 1 Running time test of VRF algorithm

VRF 算法	实验次数	总时间/ms	平均时间/ms
生成部分	1 000	1 009.832 7	1.009 8
验证部分	1 000	1 304.187 2	1.304 2

可以看出 VRF 算法的运行时间对 CRPBFT 算法的共识过程几乎没有影响,当发生视图转换需要使用可验证随机函数重新选取主节点时,可以忽略对共识时延的影响。

其次对四种算法的共识时延进行测试,设置实验节点总数 n 分别为 4、6、8、10、12、14、16、18、20,在系统中拜占庭节点数量 f 符合 $f \leq (n-1)/3$ 的前提下,进行 300 次实验并取共识时延的平均值,结果对比如图 7 所示。

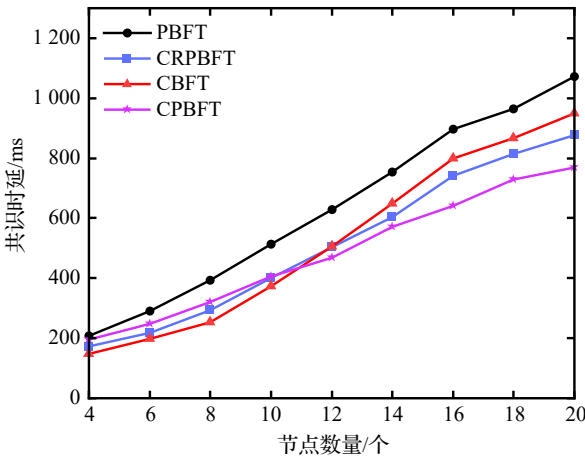


图7 共识时延测试
Fig.7 Test of consensus delay

根据图 7 可以看出,随着节点数量的增加,四种算法的共识时延都逐渐增大。不同节点数量下 PBFT 的共识时延是最高的,CRPBFT 算法整体低于 PBFT 算法,是

因为CRPBFT算法选取的主节点信誉值较高,主节点身份不易被恶意预测攻击,共识节点集群更为可信,不会导致频繁的视图转换,具有较低的共识时延和较高的共识效率。在节点数量较少时,由于CRPBFT算法增加了一致性哈希分组策略,因此共识时延略高于CBFT算法,而当节点数量大于12时,恶意节点逐渐被淘汰出CRPBFT算法的共识集群,共识节点数量减少,因此共识时延要低于CBFT算法。当节点数量较多时,由于CPBFT算法对没有拜占庭节点时的共识阶段进行了简化,因此CRPBFT算法的时延略高于CBFT算法。

CRPBFT算法的信誉计算模型相较于CBFT算法和CPBFT算法的奖惩方式更加灵活,对信誉奖励积累逐渐减缓,能够激发新加入节点的积极性。因此从系统的长期运行来看,CRPBFT算法更有利于维护良性的共识环境,提升共识效率。

3.2 吞吐量

吞吐量反映了区块链系统在单位时间内处理交易事务的能力,通常使用每秒交易处理量TPS(transaction per second)表示系统在一定时间内成功提交有效交易的速率。式(8)为TPS计算公式,其中 $time$ 表示从交易创建到区块上链所花费的时间, $Trans_{time}$ 为出块时间内系统所提交的交易数量。

$$TPS = \frac{Trans_{time}}{time} \quad (8)$$

本节实验中通过客户端发送1000次交易请求来测试四种算法的吞吐量,取多次实验数据的平均值来保证实验结果的一般性。

由图8可看出随着节点数量的不断增加,四种算法的吞吐量都呈下降的趋势。当节点数量较少时,PBFT算法中选取主节点为拜占庭节点的可能性相对较低,CRPBFT算法的信誉选主机制对吞吐量的影响效果较小,因此PBFT和CRPBFT的吞吐量在刚开始差距较小。随着节点数量的增加,CRPBFT算法能很大程度

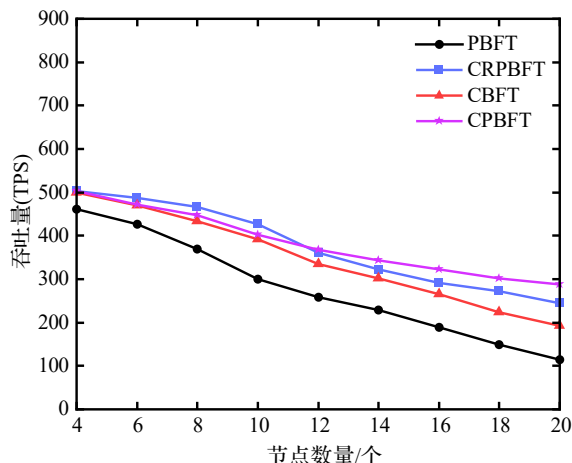


图8 吞吐量测试

Fig.8 Throughput test

上避免主节点作恶,减少节点动态变化引起的资源开销,其吞吐量的下降趋势较PBFT更为稳定,在拜占庭节点较多的情况下具有更明显的优势。相比CBFT算法和CPBFT算法,CRPBFT算法的吞吐量高于CBFT算法,因为CRPBFT算法的信誉更新由各组监督节点负责,而CBFT算法中由主节点负责信誉更新,增加了单个主节点的数据传输量。CPBFT算法在节点数量较少时吞吐量低于CRPBFT算法,当节点数量增多时,由于其对共识阶段进行了简化,所以吞吐量略高于CRPBFT算法。

3.3 系统稳健性

稳健性是衡量区块链系统中共识算法性能的指标之一,在存在拜占庭节点的系统中可通过视图转换及节点成功完成交易请求的次数来表示系统的稳定程度。

引起视图转换的原因主要有网络问题、主节点作恶或被恶意节点攻击等,若系统视图转换较为频繁,则证明系统稳健性较差。在节点总数为60的系统中,分别设置不同数量的拜占庭节点,图9为一定数量拜占庭节点的系统中四种算法的视图转换情况。

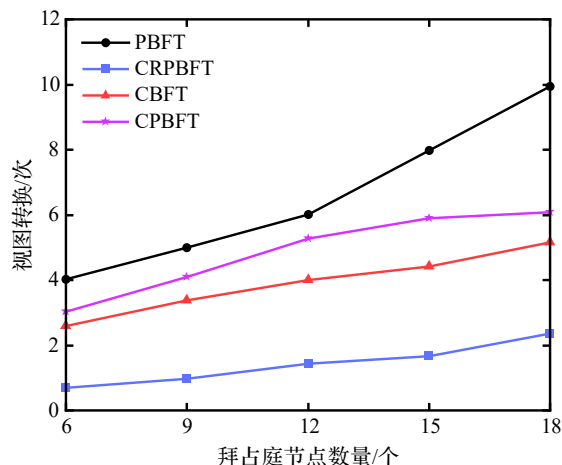


图9 视图转换次数

Fig.9 View conversion times

由图9可以看出,PBFT算法在拜占庭节点数量增加的情况下,其视图转换的次数增长较快。CBFT算法的视图转换次数整体上要小于CPBFT算法,这是由于CBFT算法在主节点出现恶意行为时直接将其替换出共识集群,而CPBFT算法对恶意主节点进行信誉惩罚,如果主节点作恶后的信誉值仍符合主节点选取的信誉级别范围,则该节点有可能再次被选为主节点,导致共识失败。CRPBFT算法在不同拜占庭节点数量下的视图转换次数均最少,即使在拜占庭节点较多的情况下也能保持极低的视图转换次数。这是因为CRPBFT算法可以通过动态的信誉机制及可验证随机函数从高信誉值节点中随机选取主节点,极大地降低了主节点为拜占庭节点的可能性,避免恶意节点预测

主节点身份发起攻击导致的视图转换,增强了系统的稳健性。

其次对系统节点成功完成交易请求的次数进行测试,对比四种算法在选取主节点后的共识过程中有效处理交易请求的能力,通过节点顺利完成交易请求的次数可以表示出共识节点集群的可靠程度。如图10所示, PBFT有效处理交易请求的次数随着节点的增加呈明显下降趋势,表明PBFT算法的系统稳健性较差。CRPBFT算法、CBFT算法以及CPBFT算法由于设置了信誉机制,其共识集群更为稳定,因此下降趋势较缓。而CBFT算法和CPBFT算法利用视图编号选取主节点,主节点的身份容易被预测攻击,因此完成交易请求的次数均低于CRPBFT算法。结果表明CRPBFT算法共识的稳定性较强,系统不仅能够区分可信节点及恶意节点,而且随机选取主节点的方式也避免了主节点身份被预测,有效降低了恶意节点对共识的影响,激发了节点的有效性,提高了系统稳健性和共识效率。

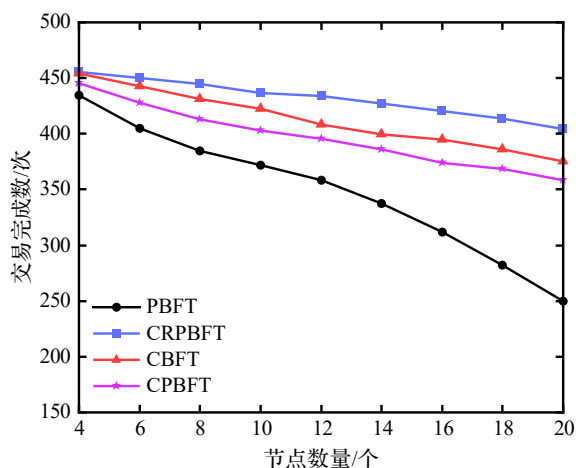


图10 交易请求有效处理次数

Fig.10 Effective processing times of transaction request

4 结束语

本文针对联盟链中主流的PBFT算法在主节点选取策略以及系统网络结构上存在的不足进行改进,提出了CRPBFT算法。对节点进行一致性哈希分组,解决系统中节点的动态变化问题,赋予系统动态性,结合信誉机制及可验证随机函数优化主节点的选取,选取出的主节点可靠性更高且难以被恶意预测。实验结果表明,CRPBFT算法提高了系统的稳健性及共识效率。CRPBFT算法的共识过程基于PBFT算法,当系统节点数量过多时,节点间的相互通信会造成通信开销迅速增长,影响系统的可扩展性。因此在之后的研究工作中,将对共识过程进行具体研究,使所提算法适应于更大规模的区块链系统。

参考文献:

- [1] NAKAMOTO S. Bitcoin: a peer-to-peer electronic cash system[J]. Decentralized Business Review, 2008, 4(2): 21260-21274.
- [2] 蔡晓晴, 邓尧, 张亮, 等. 区块链原理及其核心技术[J]. 计算机学报, 2021, 44(1): 84-131.
CAI X Q, DENG Y, ZHANG L, et al. The principle and core technology of blockchain[J]. Chinese Journal of Computers, 2021, 44(1): 84-131.
- [3] 曾诗钦, 霍如, 黄韬, 等. 区块链技术研究综述: 原理、进展与应用[J]. 通信学报, 2020, 41(1): 134-151.
ZENG S Q, HUO R, HUANG T, et al. Survey of blockchain: principle, progress and application[J]. Journal on Communications, 2020, 41(1): 134-151.
- [4] 刘双印, 雷墨鹭兮, 王璐, 等. 区块链关键技术及存在问题研究综述[J]. 计算机工程与应用, 2022, 58(3): 66-82.
LIU S Y, LEI M Y X, WANG L, et al. Survey of blockchain key technologies and existing problems[J]. Computer Engineering and Applications, 2022, 58(3): 66-82.
- [5] 夏清, 窦文生, 郭凯文, 等. 区块链共识协议综述[J]. 软件学报, 2021, 32(2): 277-299.
XIA Q, DOU W S, GUO K W, et al. Survey on blockchain consensus protocol[J]. Journal of Software, 2021, 32(2): 277-299.
- [6] JAKOBSSON M, JUELS A. Proofs of work and bread pudding protocols[C]//Communications and Multimedia Security IFIP TC6/TC11 Joint Working Conference on Communications and Multimedia Security(CMS'99), Netherlands, Sep 20-21, 1999. Boston: Kluwer Academic Publishers, 1999: 258-272.
- [7] KING S, NADAL S. PPcoin: peer-to-peer crypto-currency with proof-of-stake[J]. Connections, 2012, 19(1): 128-136.
- [8] DANIEL L. Delegated proof-of-stake whitepaper[J]. IEEE Access, 2014, 2(7): 1099-1109.
- [9] 邓小鸿, 王智强, 李娟, 等. 主流区块链共识算法对比研究[J]. 计算机应用研究, 2022, 39(1): 1-8.
DENG X H, WANG Z Q, LI J, et al. Comparative research on mainstream blockchain consensus algorithms[J]. Application Research of Computers, 2022, 39(1): 1-8.
- [10] 孙海峰, 张文芳, 王小敏, 等. 基于门限和环签名的抗自适应攻击拜占庭容错共识算法[J]. 自动化学报, 2023, 49(7): 1471-1482.
SUN H F, ZHANG W F, WANG X M, et al. A robust Byzantine fault-tolerant consensus algorithm against adaptive attack based on ring signature and threshold signature[J]. Acta Automatica Sinica, 2023, 49(7): 1471-1482.
- [11] 黄保华, 屈锡, 郑慧颖, 等. 一种基于信用的拜占庭容错共识算法[J]. 信息安全学报, 2022, 22(4): 86-92.
HUANG B H, QU X, ZHENG H Y, et al. A credit-based

- Byzantine fault tolerance consensus algorithm[J]. Netinfo Security, 2022, 22(4): 86-92.
- [12] 徐治理, 封化民, 刘飏. 一种基于信用的改进PBFT高效共识机制[J]. 计算机应用研究, 2019, 36(9): 2788-2791.
- XU Z L, FENG H M, LIU B. Improved PBFT efficient consensus mechanism based on credit[J]. Application Research of Computers, 2019, 36(9): 2788-2791.
- [13] 黄冬艳, 李浪, 陈斌, 等. RBFT: 基于Raft集群的拜占庭容错共识机制[J]. 通信学报, 2021, 42(3): 209-219.
- HUANG D Y, LI L, CHEN B, et al. RBFT: a new Byzantine fault-tolerant consensus mechanism based on raft cluster[J]. Journal on Communications, 2021, 42(3): 209-219.
- [14] 卢丽, 孙林夫, 邹益胜. 基于一致性哈希环多主节点的改进实用拜占庭容错算法[J]. 计算机集成制造系统, 2023, 29(1): 25-35.
- LU L, SUN L F, ZOU Y S. Improved PBFT algorithm of multi-primary-node based on ring of consistent hash[J]. Computer Integrated Manufacturing Systems, 2023, 29(1): 25-35.
- [15] YANG J, JIA Z, SU R, et al. Improved fault-tolerant consensus based on the PBFT algorithm[J]. IEEE Access, 2022, 10(2): 30274-30283.
- [16] LAO L, DAI X, XIAO B, et al. G-PBFT: a location-based and scalable consensus protocol for IOT-Blockchain applications[C]//2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS), New Orleans, May 18-22, 2020. Piscataway: IEEE, 2020: 664-673.
- [17] 陈宇, 贾连兴. 面向无人机集群的双层分组拜占庭容错算法[J]. 通信学报, 2022, 43(1): 96-103.
- CHEN Y, JIA L X. Two-layer grouped Byzantine fault tolerance algorithm for UAV swarm[J]. Journal on Communications, 2022, 43(1): 96-103.
- [18] TONG W, DONG X, ZHENG J. Trust-pbft: a peertrust-based practical Byzantine consensus algorithm[C]//2019 International Conference on Networking and Network Applications (NaNA), Daegu, Oct 10-13, 2019. Piscataway: IEEE, 2019: 344-349.
- [19] 王群, 李馥娟, 倪雪莉, 等. 区块链共识算法及应用研究[J]. 计算机科学与探索, 2022, 16(6): 1214-1242.
- WANG Q, LI F J, NI X L, et al. Survey on blockchain consensus algorithms and application[J]. Journal of Frontiers of Computer Science and Technology, 2022, 16(6): 1214-1242.
- [20] LEI Y, ZHAO X F, YAN J, et al. Low powered blockchain consensus protocols based on consistent hash[J]. Frontiers of Information Technology & Electronic Engineering, 2019, 20(10): 1361-1378.
- [21] MICALI S, RABIN M, VADHAN S. Verifiable random functions[C]//40th Annual Symposium on Foundations of Computer Science, New York, Oct 17-19, 1999. Piscataway: IEEE, 1999: 120-130.